

A device fingerprint or machine fingerprint is information collected about the software and hardware of a remote computing device for the purpose of identification. The information is usually assimilated into a brief identifier using a fingerprinting algorithm. One of the most common types of such is a browser fingerprint, which depends on information collected specifically by interaction with the web browser of the device.

Device fingerprints can be used to fully or partially identify individual devices even when persistent cookies (and zombie cookies) cannot be read or stored in the browser, the client IP address is hidden, or one switches to another browser on the same device. This may allow a service provider to detect and prevent identity theft and credit card fraud, but also to compile long-term records of individuals' browsing histories (and deliver targeted advertising or targeted exploits) even when they are attempting to avoid tracking—raising a major concern for internet privacy advocates.

History

Basic web browser configuration information has long been collected by web analytics services in an effort to measure real human web traffic and discount various forms of click fraud. Since its introduction in the late 1990s, client-side scripting has gradually enabled the collection of an increasing amount of diverse information, with some computer security experts starting to complain about the ease of bulk parameter extraction offered by web browsers as early as 2003.

In 2005, researchers at the University of California, San Diego showed how TCP timestamps could be used to estimate the clock skew of a device, and consequently to remotely obtain a hardware fingerprint of the device.

In 2010, the Electronic Frontier Foundation (EFF) launched a website where visitors could test their browser fingerprint. After collecting a sample of 470,161 fingerprints, they measured at least 18.1 bits of entropy possible from browser fingerprinting, but that was before the

advancements of canvas fingerprinting, which claims to add another 5.7 bits. Panopticlick, a tool run by EFF, showed that 83.6% of fingerprints were unique, reaching 94.2% when combined with Flash or Java.

In 2012, Keaton Mowery and Hovav Shacham, researchers at University of California, San Diego, showed how the HTML5 canvas element could be used to create digital fingerprints of web browsers.

In 2013, at least 0.4% of Alexa top 10,000 sites were found to use fingerprinting scripts provided by a few known third parties.

In 2014, 5.5% of Alexa top 10,000 sites were found to use canvas fingerprinting scripts served by a total of 20 domains. The overwhelming majority (95%) of the scripts were served by AddThis, which started using canvas fingerprinting in January that year, without the knowledge of some of its clients.

In 2015, a feature to protect against browser fingerprinting was introduced in Firefox version 41, but it has since been left in an experimental stage, not enabled by default. The same year, a feature named Enhanced Tracking Protection was introduced in Firefox version 42 to protect against tracking during private browsing by blocking scripts from third-party domains found in the lists published by Disconnect Mobile.

In 2016, an AmIUnique study found that 89.4% of fingerprints were unique, and attributes used to identify them were constantly evolving.

At WWDC 2018, Apple announced that Safari on macOS Mojave "presents simplified system information when users browse the web, preventing them from being tracked based on their system configuration." A 2018 study, Hiding in the Crowd, found that only 33.6% of fingerprints were unique; on mobile devices the difference was even larger, as 18.5% of mobile fingerprints were unique compared to 81% from earlier data. This study also showed that fingerprints on desktop computers are mostly unique due to combinations of attributes, while mobile devices present attributes with unique values.

In 2019, starting from Firefox version 69, Enhanced Tracking Protection was turned on by default for all users during standard non-private browsing. The feature was first introduced to protect private browsing in 2015 and was extended to standard browsing as an opt-in feature in 2018.

Diversity and stability

Motivation for the device fingerprint concept stems from the forensic value of human fingerprints.

In order to uniquely distinguish devices over time through their fingerprints, the fingerprints must be both sufficiently diverse and sufficiently stable. In practice, neither diversity nor stability is fully attainable, and improving one tends to adversely impact the other. For example, the assimilation of an additional browser setting into the browser fingerprint would usually increase diversity, but it would also reduce stability: if a user changes that setting, the browser fingerprint changes as well. However, in the absence of user opposition, fingerprints are not difficult to identify, especially since they can be based on a wide variety of data. For example, research indicates that 56.86% of users have unique extensions, and 34% of the population can be identified by the 43 characters of the fonts used.

A certain degree of instability can be compensated for by linking together fingerprints that, although partially different, likely belong to the same device. This can be accomplished by a simple rule-based linking algorithm (which, for example, links together fingerprints that differ only in browser version as it increments over time) or machine learning algorithms.

Entropy is one of several ways to measure diversity.

Sources of identifying information

Applications that are locally installed on a device are allowed to gather a substantial amount of information about the software and hardware of the device, often including unique

identifiers such as the MAC address and serial numbers assigned to machine hardware. Programs that employ digital rights management (DRM) use this information for the express purpose of uniquely identifying the device.

Even if not designed to gather and share identifying information, local applications may unintentionally expose identifying information to remote parties with which they interact. The most prominent example is web browsers, which have been shown to expose diverse and stable information in amounts sufficient to allow remote identification (see § Browser fingerprinting).

Diverse and stable information can also be gathered below the application layer by leveraging the protocols used to transmit data. Sorted by OSI model layer, examples of protocols that can be utilized for fingerprinting include:

OSI Layer 7: SMB, FTP, HTTP, Telnet, TLS/SSL, DHCP

OSI Layer 5: SNMP, NetBIOS

OSI Layer 4: TCP (see TCP/IP stack fingerprinting)

OSI Layer 3: IPv4, IPv6, ICMP

OSI Layer 2: IEEE 802.11, CDP

Passive fingerprinting techniques merely require the fingerprinter to observe traffic originating from the target device, while active fingerprinting techniques require the fingerprinter to initiate connections to the target device. Techniques that require interaction with the target device over a connection initiated by the latter are sometimes referred to as semi-passive.

Browser fingerprinting

The collection of a large amount of diverse and stable information from web browsers is enabled primarily by client-side scripting languages introduced in the late 1990s. Several

open-source browser fingerprinting libraries exist, such as FingerprintJS, ImprintJS, and ClientJS, with FingerprintJS being the most actively maintained and widely replacing earlier tools.

Browser version

Browsers provide their name and version, along with compatibility information, in the User-Agent request header. Being a self-reported statement by the client, it cannot be inherently trusted for identity verification. Instead, the browser type and version can be inferred by observing quirks in behavior: for example, the order and number of HTTP header fields is unique to each browser family, and each browser family and version differs in its implementation of HTML5, CSS, and JavaScript. Such differences can be remotely tested using JavaScript. A Hamming distance comparison of parser behaviors has been shown to effectively fingerprint and differentiate a majority of browser versions.

Browser extensions

A unique combination of extensions or plugins can be added to a fingerprint directly. Extensions may also modify the behavior of other browser attributes, increasing fingerprint complexity. Adobe Flash and Java plugins were widely used to access user information prior to their deprecation.

Hardware properties

User agents may provide system hardware information, such as phone model, in the HTTP header. Properties regarding the operating system, screen size, screen orientation, and display aspect ratio can be retrieved using JavaScript to observe the results of CSS media queries.

Browsing history

A fingerprinter could previously determine which sites in a provided list a browser had visited by querying the list with JavaScript using the CSS selector `:visited`. A list of 50 popular websites was often sufficient to generate a unique browsing history profile and infer user interests. Modern web browsers have since mitigated this tracking vector.

Font metrics

Bounding box dimensions of letters differ between browsers based on anti-aliasing and font hinting configurations, which can be measured via JavaScript.

Canvas and WebGL

Canvas fingerprinting uses the HTML5 canvas element—also utilized by WebGL to render 2D and 3D graphics in a browser—to gain identifying information about the installed device driver, video card, or graphics processing unit (GPU). Canvas-based techniques may also be used to identify installed computer fonts. If no dedicated GPU is available, CPU rendering information can be extracted instead.

A canvas fingerprinting script draws text of a specified font, size, and background color. The rendered image is recovered via the `toDataURL` Canvas API method, and the resulting hashed text-encoded data serves as the user's fingerprint. Canvas fingerprinting methods produce approximately 5.7 bits of entropy. Because this technique extracts GPU-specific characteristics, the information gained is orthogonal to entropy obtained from traditional browser fingerprinting techniques such as screen resolution or JavaScript capabilities.

Hardware benchmarking

Benchmark tests can determine whether a CPU utilizes AES-NI or Intel Turbo Boost by comparing the CPU time required to execute specific simple or cryptographic algorithms.

Specialized APIs can also be leveraged, such as the Battery Status API (which constructs a short-term fingerprint based on battery charge state) or `OscillatorNode` (which can produce audio waveforms influenced by system audio processing entropy).

A hardware ID—a cryptographic hash function specified by a device vendor—can also be queried to construct a fingerprint.

Mitigation methods

Approaches exist to mitigate browser fingerprinting and enhance user privacy, though no single method completely prevents fingerprinting while maintaining full functionality for modern web applications.

Offering a simplified fingerprint

Users may attempt to reduce fingerprintability by selecting a web browser that minimizes available identifying information, such as system fonts, device IDs, canvas element rendering, WebGL data, and local IP addresses.

In 2017, Microsoft Edge was evaluated as the most fingerprintable browser, followed by Firefox, Google Chrome, Internet Explorer, and Safari. Among mobile browsers, Google Chrome and Opera Mini were found to be most fingerprintable, followed by mobile Firefox, mobile Edge, and mobile Safari.

Tor Browser disables fingerprintable features, including canvas and WebGL APIs, and alerts users to fingerprinting attempts. To reduce diversity, Tor Browser restricts webpage viewport dimensions to predefined increments, resulting in a slightly letterboxed interface.

As of 2020, major privacy-focused browsers, including Brave, Firefox, and Tor Browser, built-in anti-fingerprinting defenses, whereas default configurations of Google Chrome provided limited protection.

Offering a spoofed fingerprint

Spoofing exposed parameters (such as the User-Agent) can reduce fingerprint diversity, though poorly implemented spoofing may unintentionally make a user more identifiable if the spoofed configuration is rare.

Varying spoofed details across site visits—such as introducing subtle random noise into audio and canvas rendering—reduces fingerprint stability. This randomized fingerprinting defense was integrated into the Brave browser in 2020.

Blocking scripts

Blocking client-side scripts from third-party or first-party domains (e.g., by disabling JavaScript entirely or using extensions like NoScript) can prevent tracking, though it may break website functionality. A common approach is selective blocking of tracking domains via blocklists (used by ad blockers) or behavior analysis (used by tools like Privacy Badger).

Using multiple browsers

Using different browsers on the same machine generates distinct browser fingerprints. However, if neither browser is protected against fingerprinting, underlying hardware-level attributes may still allow cross-browser linkability.

See also

Anonymous web browsing

Browser security

Browser sniffing

CSS fingerprinting

Evercookie

Fingerprint (computing)

Internet privacy

Web tracking

References

Further reading

Fietkau, Julian (2020-12-28). "The Elephant In The Background: Empowering Users Against Browser Fingerprinting". Chaos Communication Congress 2020.

Angwin, Julia; Valentino-DeVries, Jennifer (2010-11-30). "Race Is On to 'Fingerprint' Phones, PCs". The Wall Street Journal. Retrieved 2018-07-10.

Segal, Ory; Fridman, Aharon; Shuster, Elad (2017-06-05). Passive Fingerprinting of HTTP/2 Clients (PDF) (Technical report). BlackHat Europe. Retrieved 2022-02-09.

External links

Panopticlick by the Electronic Frontier Foundation

Am I Unique by INRIA and INSA Rennes